Day 25: Weight Initialization in TensorFlow

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Generating synthetic data
X_train = np.random.rand(100, 1)
y_train = 3 * X_train + np.random.randn(100, 1) * 0.1
```

```
# Define a model with different weight initializers
def build_model(initializer):
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(64, activation='relu', input_shape=(1,), kernel_initializer=initializer),
        tf.keras.layers.Dense(1)
    ])
    return model
```

Common Weight Initialization Methods

Random Normal Initialization: Weights are drawn from a normal distribution with a small standard deviation. Xavier (Glorot) Initialization: Designed to keep the variance of gradients consistent across layers. He Initialization: Specially designed for ReLU activation to avoid vanishing/exploding gradients.

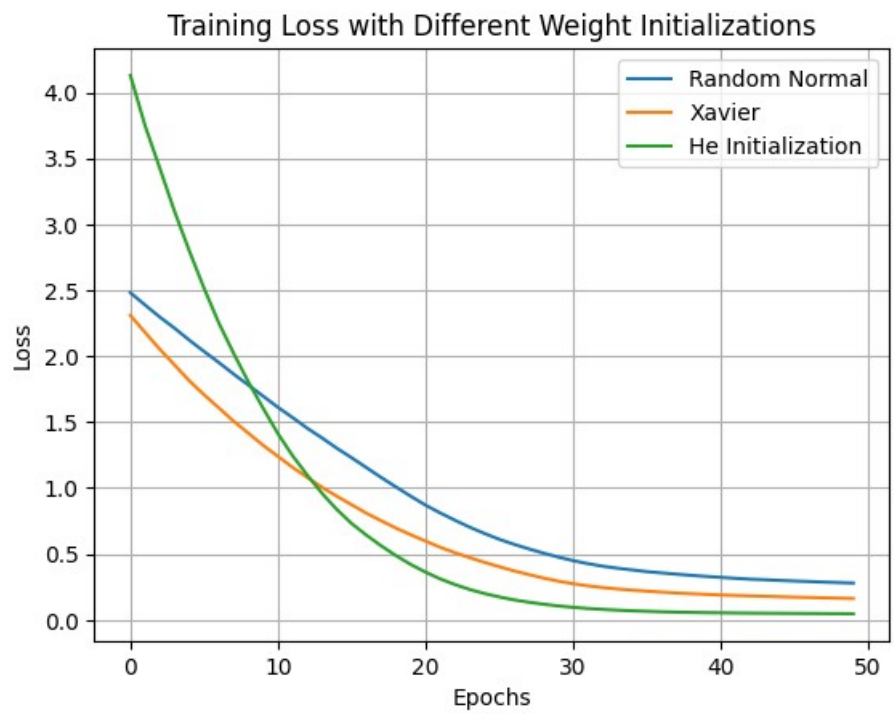
```
# Using Random Normal Initialization
model_random_normal = build_model('random_normal')
model_random_normal.compile(optimizer='adam', loss='mean_squared_error')

# Using Xavier Initialization (Glorot Uniform)
model_xavier = build_model('glorot_uniform')
model_xavier.compile(optimizer='adam', loss='mean_squared_error')

# Using He Initialization
model_he = build_model('he_normal')
model_he.compile(optimizer='adam', loss='mean_squared_error')
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
```

```
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Training the models with different weight initializers
history_random_normal = model_random_normal.fit(X_train, y_train, epochs=50, verbose=0)
history_xavier = model_xavier.fit(X_train, y_train, epochs=50, verbose=0)
history_he = model_he.fit(X_train, y_train, epochs=50, verbose=0)
```

```
# Plotting the loss curves for each initialization method
plt.plot(history_random_normal.history['loss'], label='Random Normal')
plt.plot(history_xavier.history['loss'], label='Xavier')
plt.plot(history_he.history['loss'], label='He Initialization')
plt.title('Training Loss with Different Weight Initializations')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



"""

- **Random Normal**: Can result in slower convergence due to weight values being too small.
- **Xavier Initialization**: Keeps variance across layers balanced, which helps with deep networks.
- **He Initialization**: Best suited for ReLU activations, leading to faster convergence. """

```
Start coding or generate with AI.
```